Mirage: Physically Shared, Observably Private KVCache for Disaggregated LLM Serving

Anonymous Submission

Abstract

Disaggregated LLM serving systems such as Mooncake pool the DRAM and SSD of entire GPU clusters into a deduplicated KVCache that every request can reuse, cutting prefill computation substantially. We show that this sharing is observable in ways existing defenses do not address. Beyond the known time-to-first-token channel, a cluster-scale KVCache leaks through cache-aware admission control, through billed cached-token counts, through the latency ladder of its storage tiers, and through eviction pressure that survives per-tenant key salting. Current defenses close such channels only by partitioning or selectively isolating the cache, forfeiting the reuse that justifies the architecture. Our key insight is that confidentiality requires isolating what a principal can *observe*, not what the system *stores*. MIRAGE keeps the physical KVCache global and deduplicated, but derives every client-visible quantity—first-token release time, admission decisions, and usage accounting—from a per-principal virtual cache with a fixed quota. A slack-aware scheduler converts the latency withheld from cross-tenant hits into capacity for other requests. We implement MIRAGE on vLLM and Mooncake Store and evaluate it against six attacks using Mooncake production traces. MIRAGE reduces all **[six]** channels to chance-level distinguishability while retaining **[≥90%]** of global sharing's prefill savings and serving **[X%]** more SLO-compliant requests than per-tenant isolation.

1 Introduction

Large language model (LLM) serving has become a storage problem. Every request first runs a *prefill* pass that turns its prompt into key–value (KV) tensors, and for long prompts this pass dominates cost and time-to-first-token (TTFT). Serving systems therefore cache KV tensors and reuse them whenever a new prompt shares a prefix with an old one [13, 31]. Mooncake [19], the serving platform of Moonshot AI's Kimi and a FAST'25 Best Paper, pushed this idea to cluster scale: it pools the otherwise idle CPU DRAM, SSD and RDMA NICs of a GPU cluster into a petabyte-scale, deduplicated KVCache, and schedules every request around where its reusable blocks live. On real traces this raised effective request capacity by 59%–498% over vLLM baselines [19]. The design is no longer one company's: Mooncake's store and transfer engine now back cluster-wide KV pools in vLLM, SGLang and LMCache [12, 14].

A shared cache is shared state, and shared state leaks. A line of recent work shows that prefix caching in a single serving engine creates a TTFT side channel: an attacker who guesses part of another user's prompt sees a faster first token when the guess is cached, and can reconstruct system prompts and user inputs token by token [21, 25, 32]. Defenses followed. Engines now let operators salt cache keys per tenant [23, 16]; KVGov salts at the point where prompts diverge [1]; SafeKV shares only blocks it judges non-sensitive [5]; CacheSolidarity detects suspicious reuse and isolates the affected prefixes [18].

These defenses share two limits. First, each buys confidentiality by giving up *physical* sharing, either everywhere or wherever a detector fires; in a Mooncake-style cluster, where the global pool exists precisely to raise hit rates beyond what any single node can hold, that cost lands on the system's main benefit. Second, each reasons about one engine and one channel, TTFT. A cluster-scale KVCache exposes more. Its scheduler admits or rejects requests based on predicted TTFT, which depends on the global prefix match. Commercial APIs report and discount cached tokens in every response [CITE: provider prompt-caching docs, e.g. OpenAI cached_tokens usage field], and a 2026 vLLM vulnerability turned that count into a cross-tenant membership oracle [24]. Its blocks live in tiers whose latencies form a ladder that reflects how recently and how widely a prefix was used. And its capacity is shared, so one tenant's traffic evicts another's blocks even when keys are salted.

This paper. We make two contributions. The first is an attack study of cluster-scale, tiered KVCache serving (Section 4). We build six attacks against a Mooncake deployment (**A1**–**A6**), four of which (**A2**–**A4**, **A6**) we have not found in prior work [TODO: re-run the prior-art search before submission], and show that **[N of 6]** succeed with **[AUC ≥ X]** under realistic load; the admission and accounting channels require no timing at all, and the eviction channel survives per-tenant salting.

The second is MIRAGE, a defense built on one observation: *confidentiality requires isolating what a principal can observe, not what the system stores.* MIRAGE leaves Mooncake's physical layer unchanged—blocks remain globally deduplicated, replicated and transferred over RDMA—and adds an observable layer that makes every client-visible

quantity a function of the principal's own history and a coarse public load signal. A per-principal Virtual Cache Directory (VCD) answers how long a prefix the principal would have hit in a private cache of fixed quota. A release controller holds the first token until a deadline computed from that virtual hit length; admission decisions and usage accounting use it too. Because the physical cache still hits, the GPU finishes early; a slack-aware scheduler reinvests that time in other requests. To a client, MIRAGE is indistinguishable from an isolated system; to the operator, it does the compute of a shared one.

The idea of releasing outputs on a schedule that depends only on public information has a long history in timing-channel mitigation [3, 30], and the leakage of cross-user deduplication was first studied in cloud storage [11]. MIRAGE's contribution is to apply observational isolation to a disaggregated KV store, where the right schedule comes from a *counterfactual* cache rather than from padding, where non-timing observables must be covered by the same counterfactual, and where the withheld latency can be recovered as throughput.

We implement MIRAGE on vLLM with the Mooncake Store connector and a Mooncake-style global scheduler (Section 7). On **[testbed]** with Mooncake's released traces, MIRAGE **[brings all six channels to AUC $\leq$ 0.55]**, keeps **[$\geq$90%]** of the prefill GPU time saved by unrestricted global sharing, and serves **[X%]** more SLO-compliant requests than per-tenant isolation, at a cost of **[Y%]** higher median TTFT for requests whose only hits are cross-tenant (Section 8).

Contributions.

- The first systematic study of leakage in cluster-scale, tiered KVCache serving, including channels through admission control, usage accounting, tier latency, replication and eviction (Section 4).
- Observational isolation for shared KV caches: a definition relative to a public load signal, and a design that meets it while preserving physical sharing (Sections 3, 5 and 6).
- MIRAGE, a prototype on vLLM and Mooncake Store, with a slack-aware scheduler that converts withheld latency into capacity (Sections 5 and 7).
- An evaluation against six attacks and five defenses on Mooncake production traces (Section 8). We will release code, attack harnesses and principal-labelled trace derivatives (Appendix B).

2 Background

2.1 Prefix caching

A decoder-only transformer computes, for each prompt token, key and value vectors at every layer. A token's KV depends only on itself and the tokens before it, so KV computed for one prompt is valid for any later prompt that shares the same prefix [19]. Engines store KV in fixed-size blocks and key each block by a hash chained over its own tokens and its prefix, $h_j = H(h_{j-1}, \text{tokens}_j)$; a lookup returns the longest matching chain from the first block [13, 1]. A hit skips that part of prefill, cutting TTFT roughly in proportion to the prefix length.

2.2 Mooncake: a KVCache-centric architecture

Mooncake [19] separates prefill and decode onto different node pools and turns the KVCache into a cluster-wide storage tier.

Mooncake Store. KV blocks of 16–512 tokens (256 in the paper's evaluation) live in a distributed pool built from the host DRAM and SSD of every node. Each block's key covers the block and its prefix, so identical content is deduplicated cluster-wide. Hot blocks are replicated across nodes to avoid fetch congestion, and when the pool fills, blocks are evicted LRU unless in use. A transfer engine moves blocks over multiple RDMA NICs per node; the paper reports 87 GB/s and 190 GB/s on 4×200 and 8×400 Gbps fabrics. The released store adds per-object replica counts and soft and hard pins [12].

Conductor. A global scheduler places each request (Algorithm 1 in [19]). It hashes the prompt into block keys, finds each prefill instance's prefix-match length, and estimates TTFT per instance as transfer time for any remote blocks plus queueing plus prefill time, the last from a regression fitted offline on request length and match length. It picks the instance with the lowest estimate, and if the TTFT or TBT SLO cannot be met, returns HTTP 429 instead of serving the request. Placement decisions also trigger replication of hot blocks.

Why it matters here. Three design choices that make Mooncake fast also make it observable: (i) deduplication is keyed by content alone, with no principal in the key; (ii) admission is a function of the *global* match length; and (iii) a block's physical location—local HBM, local DRAM, remote DRAM, SSD—depends on other requests' access history. The paper evaluates a single first-party service and does not discuss tenancy or confidentiality [19].

2.3 Multi-tenant serving

LLMs are mostly sold as APIs to many mutually distrusting customers. Responses expose TTFT and inter-token time (TBT) to the client, HTTP status codes, and usage metadata that typically includes the number of cached prompt tokens, which providers bill at a discount [CITE: provider docs]. Engines offer per-request `cache_salt` to separate tenants' key spaces; NVIDIA Dynamo uses the same salt to namespace

KV-aware routing [23, 16]. Salting is only as strong as its plumbing: one 2026 vLLM bug dropped the salt on tool continuations, restoring a cached-token oracle [24].

3 Threat Model and Goals

System. An operator runs a Mooncake-style cluster: disaggregated prefill and decode pools, a pooled DRAM/SSD KV store with content-keyed deduplication, and a cache-aware global scheduler. Clients are *principals*—API keys or tenants—that the operator authenticates. The operator, the serving software and the hardware are trusted.

Attacker. The attacker controls one or more principals, possibly many (Sybils). It knows the model, block size, scheduling algorithm and approximate hardware, and may submit arbitrary prompts at chosen times subject to rate limits. For each request it observes everything a client sees: the admission outcome (including HTTP 429), the arrival time of every output token measured at its client, and the usage record (prompt, cached and output token counts, and the resulting charge). Its goals are to learn whether a prefix was processed for another principal (*membership*), to reconstruct another principal's prompt (*reconstruction*), or to learn how recently or how widely a prefix is used (*activity*).

Out of scope. Attackers with code on cluster hosts or GPUs (microarchitectural channels, Rowhammer on KV memory [27]), network observers of encrypted token streams, a malicious operator, semantic caches, non-prefix chunk reuse [28], information the model itself memorizes, and denial of service. Contention in the *decode* batch affects TBT in any shared system, isolated caches included; we treat it as part of the public load signal and do not claim to remove it.

3.1 Observations and the public load signal

For a request r from principal p, let $o(r) = (a_r, \tau_r, \mathbf{t}_r, u_r)$ be its admission outcome, first-token release time relative to arrival, later token times, and usage record. The attacker's view is the sequence $\mathscr{O}_p$ of $o(r)$ over its requests.

Some dependence on others is unavoidable: if the cluster is busy, everyone waits longer. MIRAGE makes this dependence explicit through a *public load signal* $\mathscr{L}$: a load level drawn from k values, published once per epoch of length Δ. Every scheduling quantity that a client could observe may depend on $\mathscr{L}$, and on nothing else outside the principal's own requests.

Definition 1 (Observational isolation)**.** *A serving system is* observationally isolated *relative to $\mathscr{L}$ if, for every principal p and any two executions in which p submits the same requests at the same times and $\mathscr{L}$ takes the same values, the deterministic part of $\mathscr{O}_p$ is identical, and the remaining randomness in $\mathscr{O}_p$ is independent of every other principal's requests.*

An isolated cache with a fixed per-principal quota meets this definition but loses all cross-principal reuse. Through $\mathscr{L}$, the definition admits leakage of at most $\log_2 k$ bits per epoch about *aggregate* load, never about a specific prefix.

3.2 Goals

G1 Confidentiality. Observational isolation for admission, first-token time and usage, with every violation (a missed release deadline, Section 5) detected, counted and bounded.

G2 Preserved sharing. Physical reuse equal to unrestricted global sharing: no block is recomputed because of MIRAGE.

G3 Capacity. More SLO-compliant requests than per-principal isolation on the same hardware.

G4 Deployability. No change to the model, kernels or KV store; changes confined to the scheduler and the API front end.

4 Leakage in Cluster-Scale KVCache

We first ask what an API tenant can learn from a Mooncake-style deployment, under both unrestricted sharing and per-principal salting. Table 1 summarizes the six attacks. **A1** and **A5** adapt known single-engine channels [21, 24] to the cluster; **A2**, **A3**, **A4** and **A6** arise from Mooncake's global pool, tiers, replication and admission control.

4.1 Setup and method

[TODO: Finalize after week-1 testbed. Planned: **[2]** nodes $\times$ 8 **[GPU model]**, **[2×200 Gbps]** RoCE, vLLM **[version]** with the Mooncake Store connector, a Conductor-style router implementing Algorithm 1 of [19], models **[Llama-3.1-8B, Qwen 14B, 70B-class]**.] Background traffic replays the conversation and tool&agent traces released with Mooncake [19] at **[three]** load levels. Victims send prompts from templated sensitive corpora (system prompts; prompts embedding names and medical or financial attributes, following [5, 18]). For each attack we train a classifier on attacker-only calibration traffic and report AUC, attack success rate and queries per recovered token. We also measure how load erodes each channel, since contention can make quiet-system measurements optimistic [4].

4.2 Timing channels

A1: Global hit. The attacker sends a candidate prefix. If any principal's request placed matching blocks anywhere in the pool, the Conductor routes the request to fetch them and TTFT drops by the saved prefill minus the RDMA transfer. Unlike single-engine attacks, the victim and attacker need not share a node. **[Result: AUC, success rate per token, effect of prefix length.]**

A2: Tier ladder. A hit is not binary. A block may sit in the serving node's HBM or DRAM, in another node's DRAM, or

	Channel	Mechanism	Attacker learns	Timing?	Survives salting?	Result
A1	Global hit	Cross-node block fetch beats recompute	Membership	yes	no	**[AUC]**
A2	Tier ladder	HBM / local DRAM / remote DRAM / SSD latency classes	Recency	yes	no	**[AUC]**
A3	Replication	Replica count changes fetch contention	Popularity	yes	no	**[AUC]**
A4	Admission	Hit admitted, miss gets HTTP 429 near the SLO edge	Membership	**no**	no	**[AUC]**
A5	Accounting	Usage record reports cached tokens	Exact hit length	**no**	no†	**[AUC]**
A6	Eviction	Victim traffic evicts attacker-primed blocks	Activity volume	yes	**yes**	**[AUC]**

Table 1: Attacks on a Mooncake-style deployment. "Survives salting" means the channel persists when every principal has its own cache key namespace. †Unless salting is bypassed, as in [24]. Results: **[distinguishability (AUC) at moderate load, Section 8.2]**.

PLACEHOLDER

TTFT CDFs for a 4k-token prefix: miss vs. hit from local HBM, local DRAM, remote DRAM, SSD, at low and moderate load.

Figure 1: **A2**: the tier ladder. **[Takeaway sentence with numbers.]**

on SSD, each with a distinct load time; which tier holds it depends on how recently and where it was last used. Probing a known prefix and classifying the latency class reveals *when* it was last touched, not only *whether*. **[Result: per-tier TTFT distributions (Figure 1), classification accuracy.]**

A3: Replication. Mooncake replicates blocks that many requests fetch [19]. More replicas mean less contention on any one holder, so repeated probes of a popular prefix see lower and less variable fetch times. The attacker estimates how many principals use a prefix—for example, how much traffic a competitor's agent receives. **[Result: correlation between estimated and true request rate.]**

4.3 Non-timing channels

A4: Admission. The Conductor rejects a request when predicted TTFT exceeds the SLO, and the prediction uses the global prefix match. The attacker raises its prompt length until, at the current load, a miss would be rejected and a hit would not; HTTP 429 versus 200 then answers the membership question with no clock at all, immune to network jitter. **[Result: success rate versus load, number of calibration queries.]**

A5: Accounting. Usage records that report cached tokens give the exact hit length per request. With global sharing this is a perfect membership oracle; with salting it leaks whenever the salt is lost along any code path [24]. **[Result: confirm on testbed; survey of what public APIs report, with disclosure (Appendix A).]**

4.4 A contention channel that survives salting

A6: Eviction. Salting separates key spaces but not capacity. The attacker primes long, unique prefixes of its own, waits, and re-probes them. Global LRU evicts the attacker's blocks only when others insert enough new blocks, so the fraction of the attacker's prefixes that survive measures other principals' cache insertion volume over the interval, with time resolution set by the probe rate. This is a Prime+Probe analogue at the storage tier [29]. **[Result: correlation between surviving fraction and victim insertion rate; salted vs. unsalted.]**

4.5 Takeaways

[TODO: Write once results exist. Expected shape: (1) the global pool widens membership attacks across nodes; (2) the tiers turn membership into recency and popularity; (3) admission and accounting leak with no timing, so timing-only defenses such as padding cannot close them; (4) salting removes **A1–A5** at the price of all cross-principal reuse, and still leaves **A6**.]

5 MIRAGE Design

MIRAGE separates two questions that Mooncake answers with one number. *How much prefill can we skip?* is answered by the physical, global prefix match p^{phys}, exactly as in Mooncake. *What may this principal observe?* is answered by a virtual match $p^{\text{virt}} \le p^{\text{phys}}$: the hit length the principal would have seen with a private cache of quota Q. Figure 2 shows the resulting two layers. Everything in the observable layer depends only on the principal's own history and $\mathscr{L}$; everything in the physical layer is free to share.

5.1 Virtual Cache Directory

The VCD holds, for each principal p, an LRU list of block keys with total size at most Q tokens. It stores no tensors. For a request with block chain $h_1, \ldots, h_m$, p^{virt} is the largest j such that $h_1, \ldots, h_j$ are all in p's list or in the *declared-public* set (below). After the request, its blocks are inserted at the

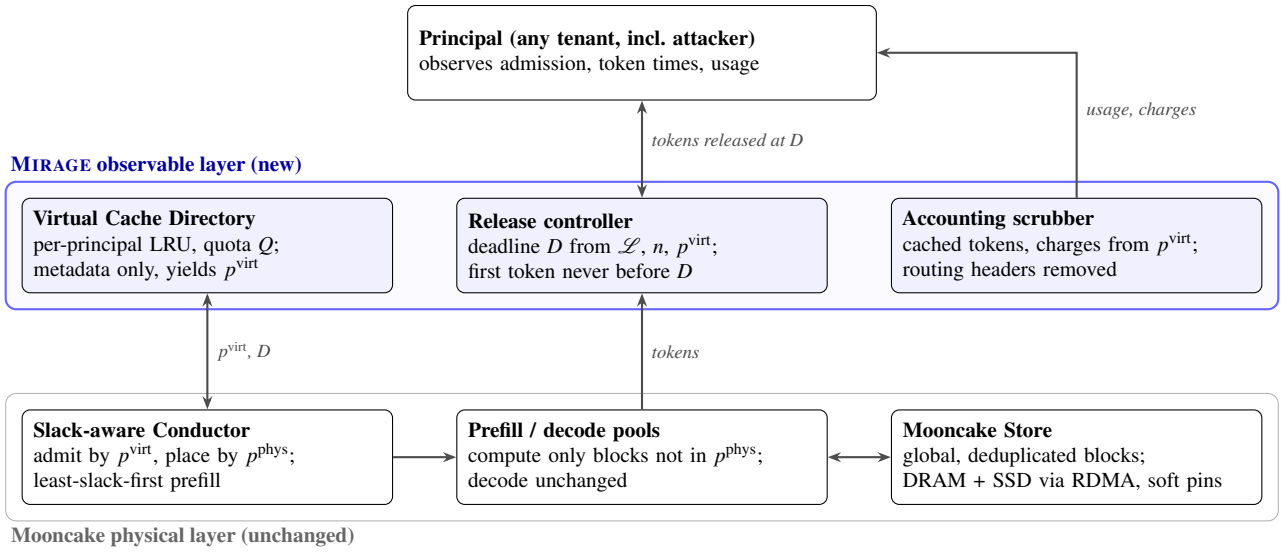


Figure 2: MIRAGE keeps Mooncake's physical layer unchanged and derives everything a principal can observe from its own virtual cache. p^{virt}: virtual (private) prefix-hit length; p^{phys}: physical (global) hit length; D: release deadline; $\mathscr{L}$: public load signal.

head of p's list, evicting from the tail. At 256 tokens per block, a 1M-token quota is about 3.9k entries, a few hundred KB per active principal **[measured overhead, Section 8.6]**.

Declared-public prefixes. Much valuable reuse is of content that is not secret: the operator's own system prompts, popular public documents, tool schemas. The operator may register such prefixes in a public set that every principal's VCD treats as present. Because membership in the set is fixed by declaration rather than by traffic, a hit on it reveals nothing about other principals. [TODO: Decide: also allow principals to opt into shared groups (e.g., one organization), which share a single virtual list.]

5.2 Release controller

For each admitted request r with n prompt tokens arriving at t_r, the release controller computes

$$D_r = t_r + \left\lceil \hat{q}(\mathscr{L}) + \hat{T}(n, p^{\text{virt}}) \right\rceil_{\beta}, \tag{1}$$

where $\hat{T}$ is the prefill-time model the Conductor already keeps [19], $\hat{q}$ is a queueing allowance fixed per load level, and $\lceil\cdot\rceil_{\beta}$ rounds up to a bucket of width β. The first token is released at $\max(D_r, f_r)$, where f_r is when it is actually ready; later tokens keep their measured spacing after it. Every input to Equation (1) is either the principal's own or public.

Misses. If $f_r > D_r$, the release time depends on physical state and may leak. MIRAGE counts every miss, and, following predictive mitigation [3, 30], enlarges the principal's bucket for the rest of the epoch after a miss, so that an attacker cannot convert misses into a high-rate channel. Section 6 bounds the resulting leakage; Section 5.5 keeps misses rare.

5.3 Admission and accounting

Mooncake rejects a request when its predicted TTFT exceeds the SLO, using the global match. MIRAGE makes the same decision from $D_r - t_r$, so a request is admitted if and only if it would be admitted by an isolated system at the same load level. The operator still needs protection from real overload; MIRAGE gets it by setting the per-epoch admission budget from measured capacity and publishing the result as part of $\mathscr{L}$, rather than by consulting physical cache state per request.

The usage record reports p^{virt} as cached tokens and bills the discount on p^{virt}. The difference $p^{\text{phys}} - p^{\text{virt}}$ is compute the operator saves but does not pass on, which is also what pays for MIRAGE. Response headers that identify instances or timing internals are removed.

5.4 Slack-aware scheduling

Because $p^{\text{phys}} \geq p^{\text{virt}}$, a request often needs less GPU time than its deadline assumes. Finishing it early gains nothing—its first token waits for D_r—so MIRAGE instead spends that slack on other requests. The Conductor computes each request's latest start time $s_r = D_r - \hat{T}(n, p^{\text{phys}}) - \hat{x}_r - \varepsilon$, where $\hat{x}_r$ is the estimated block-transfer time and ε a safety margin, and runs the prefill queue least-slack-first (Algorithm 1). Placement keeps Mooncake's cache affinity, but its objective changes from "lowest TTFT" to "least GPU time that still meets D_r". Starting a request just in time, rather than early,

Algorithm 1 MIRAGE Conductor (changes to Mooncake's Algorithm 1 in **bold**)

Require: request r from principal p; prefill pool P; load level $\mathscr{L}$
1: $keys \leftarrow \text{PREFIXHASH}(r.tokens, B)$
2: $\mathbf{p}^{\text{virt}} \leftarrow \text{VCD.MATCH}(\mathbf{p}, \mathbf{keys})$ ▷ own history + public set
3: $\mathbf{D_r} \leftarrow \mathbf{t_r} + \lceil \hat{\mathbf{q}}(\mathscr{L}) + \hat{\mathbf{T}}(\mathbf{n}, \mathbf{p}^{\text{virt}}) \rceil_\beta$
4: **if** $\mathbf{D_r} - \mathbf{t_r} > SLO_{\text{TTFT}}$ **or budget**$(\mathscr{L})$ **exhausted then**
5: **return** REJECT ▷ same decision an isolated system makes
6: **end if**
7: **for** $i \in P$ **do**
8: $p_i^{\text{phys}} \leftarrow$ prefix match on i (local or fetchable)
9: $w_i \leftarrow \hat{T}(n, p_i^{\text{phys}}) + \hat{x}_i$ ▷ GPU time + transfer
10: $\mathbf{s_i} \leftarrow \mathbf{D_r} - \hat{\mathbf{q}}_\mathbf{i} - \mathbf{w_i} - \varepsilon$ ▷ latest start
11: **end for**
12: $i^\star \leftarrow \arg\min_{i: s_i \geq now} w_i$ ▷ **least work that meets** D_r
13: enqueue r on $i^\star$ ordered by $s_{i^\star}$ ▷ **least slack first**
14: $\text{VCD.INSERT}(p, keys)$; $\text{ORC.HOLD}(r, D_r)$

also avoids holding its KV in decode memory while the first token waits.

5.5 When the virtual cache hits and the physical cache does not

The VCD may say a block is present after global LRU has evicted it. The request must then recompute it and may miss D_r. MIRAGE soft-pins, through Mooncake Store's per-object pin interface [12], the physical blocks referenced by recently used VCD entries of active principals, up to a pin budget, and raises the priority of requests whose physical work exceeds their virtual estimate. **[Pin budget and miss rate, Section 8.3.]** [TODO: Check whether a pin budget smaller than the sum of active quotas suffices on the traces; if not, quotas must shrink.]

5.6 What MIRAGE costs

Relative to unrestricted sharing, a request whose only hits are on other principals' blocks now sees the TTFT of a miss: exactly what per-principal isolation would give it. Requests that hit their own history or the public set are unaffected. MIRAGE adds no prefill work, by construction.

6 Security Analysis

[TODO: The statements below are proof sketches to be turned into full proofs (or a mechanized model) before submission. Nothing here is claimed as proved yet.]

6.1 Isolation outside misses

Proposition 1 (sketch)**.** *Consider any two executions in which principal p submits the same requests at the same times and $\mathscr{L}$ takes the same values. If no request of p misses its deadline in either execution, then p's admission outcomes, first-token release times and usage records are identical in both.*

Sketch. By induction over p's requests. p's VCD list is updated only by p's own requests (Section 5.1), and the public set is fixed, so p^{virt} is the same in both executions. The deadline in Equation (1), the admission test in Algorithm 1 and the usage record are functions of $(t_r, n, p^{\text{virt}}, \mathscr{L})$ only. Without a miss, release happens at D_r. Tokens after the first are released with their measured spacing, which depends on decode-batch contention; we place that in $\mathscr{L}$'s scope (Section 3) and do not claim isolation for it. □

6.2 Bounding what misses leak

A miss releases the first token at $f_r > D_r$, and f_r can depend on physical state. Two factors limit the damage. First, misses are observable to MIRAGE, so they can be counted, rate-limited and reported. Second, after each miss MIRAGE enlarges the principal's bucket for the rest of the epoch, the penalty schedule used in predictive mitigation, for which Askarov, Zhang and Myers bound total leakage by a polylogarithmic function of elapsed time [3, 30].

Conjecture 1. *Over E epochs with M misses in total, a principal learns at most $E \log_2 k$ bits from $\mathscr{L}$ plus $O(M \log_2(T_{\max}/\beta))$ bits from misses, where $T_{\max}$ is the largest release delay.*

[TODO: Prove by adapting the predictive-mitigation argument; check that its assumptions (a public schedule, a bounded set of release times) hold for Equation (1).]

6.3 Coverage of the attacks

- **A1–A3 (timing).** Global hits, the tier that served a block and its replica count change only f_r, never D_r. Outside misses they are invisible.
- **A4 (admission).** Admission depends on p^{virt} and $\mathscr{L}$, not on the global match.
- **A5 (accounting).** Cached tokens are reported as p^{virt}.
- **A6 (eviction).** The attacker's *virtual* list is evicted only by its own requests. Physical eviction of its blocks causes recomputation and, at worst, a miss, which the pin budget makes rare and the bound above covers.
- **Sybils.** Each principal has an independent list, so many attacker principals share nothing that a single one does not.

6.4 Residual channels

The public load signal. Cross-principal hits reduce real load and so can shift $\mathscr{L}$. This channel carries at most $\log_2 k$ bits per epoch about aggregate load and none about any specific prefix; k and Δ are operator knobs (Section 8.5).

Decode contention. TBT depends on who else is decoding, as in any shared GPU deployment, isolated caches included. MIRAGE does not change it.

Mis-declared public content. A secret registered in the public set is unprotected. The set should hold only operator-owned or published content.

Outside the serving path. Dashboards, logs or support tools that expose physical hit rates per principal reopen the channel; MIRAGE's guarantee covers only the client API.

7 Implementation

[TODO: Fill in as built. Target components and sizes:]

- **Serving stack.** vLLM **[version]** with prefill/decode disaggregation and the Mooncake Store connector [12]; Mooncake Transfer Engine over RoCE.
- **Conductor.** A global router implementing Mooncake's Algorithm 1, then Algorithm 1. [TODO: Decide whether to extend an existing KV-aware router (llm-d, Dynamo) or write a standalone one; **[LoC]**.]
- **VCD.** An in-memory, per-principal LRU keyed by the same block hashes the store uses, sharded with the Conductor; **[LoC]**.
- **Release controller and accounting scrubber.** In the OpenAI-compatible API front end: holds the first streamed token until D_r, rewrites usage records, strips internal headers; **[LoC]**.
- **Pinning.** Uses Mooncake Store's soft-pin interface; **[LoC]**.
- **Attack harness.** Client-side timing with **[clock source]**, calibration and classifiers for **A1**–**A6**; **[LoC]**.

MIRAGE changes no model, kernel or KV-store code (G4).

8 Evaluation

We answer five questions: **Q1** How strong are **A1**–**A6** under realistic load (Section 8.2)? **Q2** Does MIRAGE close them, and how do existing defenses compare (Section 8.3)? **Q3** What capacity does MIRAGE keep relative to global sharing and isolation (Section 8.4)? **Q4** How sensitive is it to quotas, sharing structure, bandwidth and $\mathscr{L}$ (Section 8.5)? **Q5** What does it cost (Section 8.6)?

8.1 Setup

Hardware and models. **[Nodes, GPUs, NICs.]** Models: **[Llama-3.1-8B, a 14B–32B Qwen model, and a 70B-class model (or dummy weights at 70B shape, as Mooncake's own reproducible evaluation used [19])]**, plus **[one MLA model]** to test how smaller per-token KV changes channel strength.

Workloads. Mooncake's released conversation and tool&agent traces [19], and its synthetic mix of ShareGPT, L-Eval and LooGLE in a 1:1:1 ratio. The traces carry remapped block hashes but no principal IDs; we assign principals by **[session clustering on shared prefix chains]** and sweep the number of principals and the share of cross-principal reuse. [TODO: Validate the assignment method; it drives every capacity number.]

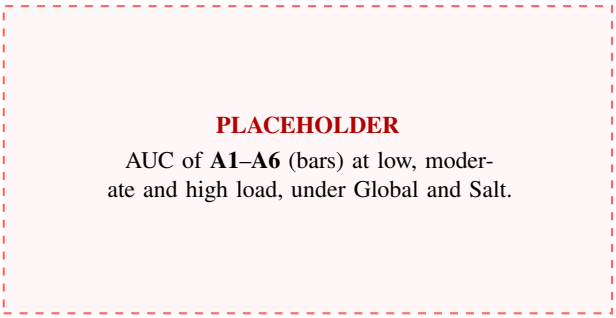


Figure 3: **[Takeaway: which channels survive load and salting.]**

Defense	A1	A2	A3	A4	A5	A6
Global	[–]	[–]	[–]	[–]	[–]	[–]
Salt	[–]	[–]	[–]	[–]	[–]	[–]
KVGov	[–]	[–]	[–]	[–]	[–]	[–]
CacheSolidarity	[–]	[–]	[–]	[–]	[–]	[–]
SafeKV	[–]	[–]	[–]	[–]	[–]	[–]
Pad	[–]	[–]	[–]	[–]	[–]	[–]
MIRAGE	[–]	[–]	[–]	[–]	[–]	[–]

Table 2: Attack AUC per defense at moderate load (0.5 = chance).

Baselines.

- **Global**: unrestricted Mooncake sharing (insecure performance ceiling).
- **Salt**: per-principal `cache_salt` [23].
- **KVGov**: salt at the divergence boundary [1].
- **CacheSolidarity**: detect-and-isolate [18], ported from its released vLLM artifact.
- **SafeKV**: sensitivity-based selective sharing [5].
- **Pad**: constant TTFT for all requests.
- **MIRAGE** and ablations: no slack reclamation; admission on p^{phys}; no accounting scrubber; no pinning.

Metrics. Security: per-channel AUC, attack success rate, queries per recovered token, deadline-miss rate. Performance: Mooncake's effective request capacity (fraction of requests meeting both TTFT and TBT SLOs) [19], prefill GPU-seconds per request, physical hit rate, TTFT p50/p99, TBT p99.

8.2 Q1: Attack strength

[Results for Table 1 and Figure 3.]

8.3 Q2: Closing the channels

[Results.] We also run an adaptive attacker that modulates load to force misses and probes $\mathscr{L}$ directly, and report misses per **[hour]** and bits leaked under Conjecture 1.

PLACEHOLDER

Effective request capacity vs. TBT SLO (as in Mooncake's Fig. 1) for Global, Salt, KVGov, CacheSolidarity and MIRAGE, conversation trace.

Figure 4: **[Takeaway: MIRAGE between Salt and Global, closer to Global.]**

8.4 Q3: Capacity

[Results: capacity, prefill GPU time, and the TTFT cost borne by requests whose only hits are cross-principal.] Ablation: without slack reclamation, how much of the gain over Salt disappears.

8.5 Q4: Sensitivity

[Sweeps:] number of principals; quota Q; fraction of cross-principal reuse; declared-public set size; network bandwidth from 25 to 400 Gbps (where Mooncake reports a cliff below 100 Gbps [19]); load-signal levels k and epoch Δ against measured leakage.

8.6 Q5: Overheads

[VCD memory per active principal and in total; Conductor CPU per request; release-controller buffering; pin budget used.]

9 Discussion and Limitations

Who pays. Under MIRAGE, cross-principal hits save the operator compute but no longer save the client latency or money. Whether that is acceptable is a product decision; the declared-public set and opt-in groups recover the reuse that is not sensitive. [TODO: Quantify how much of the trace's reuse is plausibly public (system prompts, tool schemas).]

Beyond prefix reuse. Non-prefix reuse [28], semantic caches and sparse-attention retrieval create further shared state. The VCD principle—decide observables from a private counterfactual—extends to them in principle, but the counterfactual ("would this chunk have been reusable privately?") is harder to compute. [TODO: One paragraph on non-prefix KV channels [22].]

Integrity. MIRAGE addresses confidentiality only. A shared, multi-writer pool also lets one bad writer corrupt every request that reuses a block [26, 27]; that is complementary future work.

Generality. We build on Mooncake's design, but the attacks need only a cluster-wide pool, tiers, cache-aware admission and usage reporting, which [TODO: list: LMCache remote tiers, SGLang HiCache, Dynamo KV routing] also have.

10 Related Work

KV-cache side channels. Song et al. show TTFT channels from shared KV and semantic caches and find cache sharing in commercial APIs [21]. PromptPeek reconstructs prompts by exploiting longest-prefix-match scheduling [25]; InputSnatch steals inputs through timing [32]; Gu et al. audit prompt caching across API providers [9]. Later work extends attacks to non-prefix reuse in RAG [22], studies how load degrades their reliability [4], and recovers inputs from KV contents themselves [15]. All of this targets a single engine; we study the cluster tier and its non-timing channels.

Defenses. Per-tenant salting is now built into engines [23, 16]. KVGov salts from the divergence point to keep shared preambles [1]; SafeKV shares only blocks judged non-sensitive [5]; CacheSolidarity isolates prefixes once cross-user reuse looks suspicious [18]. Each reduces physical sharing and addresses TTFT. MIRAGE keeps physical sharing and covers admission and accounting as well.

Timing-channel mitigation and deduplication. Releasing outputs on a schedule computed from public information is predictive mitigation [3, 30], itself grounded in noninterference [7]. Cross-user deduplication in cloud storage was shown to leak file existence [11], motivating proofs of ownership [10]; page deduplication and the OS page cache leak similarly [8]. Cache partitioning against co-resident attackers is long studied [20, 29]. MIRAGE's virtual cache is the counterfactual that gives predictive mitigation its schedule in a KV store.

KV-centric serving. Mooncake [19] builds on prefill/decode disaggregation [33, 17], chunked prefill [2], paged KV [13], radix-tree reuse [31] and hierarchical KV caching [6]; LMCache [14] and CacheBlend [28] broaden reuse. None of these considers tenants.

11 Conclusion

Cluster-scale KV caches turn prefix reuse from a node optimization into shared infrastructure, and with it into a shared leak that reaches beyond timing to admission, billing and eviction. MIRAGE shows that the fix need not give up the sharing: by deciding what each principal may observe from a private, counterfactual cache while the physical cache stays global, it makes a Mooncake-style system look isolated to its clients and shared to its operator. **[One sentence with the headline numbers.]**

References

[1] Tejasvi C. Addagada. Governing the KV cache: Preventing timing side-channel leakage in multi-tenant LLM inference. arXiv:2608.09225, 2026.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.

[3] Aslan Askarov, Danfeng Zhang, and Andrew C. Myers. Predictive black-box mitigation of timing channels. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, 2010. VERIFY pages.

[4] Rana Abu Bakar. Characterizing contention-induced reliability collapse in KV-cache timing side channels for multi-tenant LLM serving. arXiv:2609.06853, 2026.

[5] K. Chu, Z. Lin, D. Xiang, Z. Shen, J. Su, C. Chu, Y. Yang, W. Zhang, W. Wu, and W. Zhang. Selective KV-cache sharing to mitigate timing side-channels in LLM inference. arXiv:2508.08438, 2025. VERIFY: expand initials to full names; v2 revised February 2026.

[6] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.

[7] Joseph A. Goguen and José Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, 1982. VERIFY pages.

[8] Daniel Gruss, Erik Kraft, Trishita Tiwari, Michael Schwarz, Ari Trachtenberg, Jason Hennessey, Alex Ionescu, and Anders Fogh. Page cache attacks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019. VERIFY author list and pages (preprint arXiv:1901.01161).

[9] Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. Auditing prompt caching in language model APIs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[10] Shai Halevi, Danny Harnik, Benny Pinkas, and Alexandra Shulman-Peleg. Proofs of ownership in remote storage systems. In *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)*, 2011. VERIFY pages.

[11] Danny Harnik, Benny Pinkas, and Alexandra Shulman-Peleg. Side channels in cloud services: Deduplication in cloud storage. *IEEE Security & Privacy*, 8(6), 2010. VERIFY pages.

[12] kvcache-ai. Mooncake: A KVCache-centric disaggregated architecture for LLM serving (source repository). `https://github.com/kvcache-ai/Mooncake`. Accessed October 2026.

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[14] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, et al. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. arXiv:2510.09665, 2025. VERIFY: expand initials to full names.

[15] Zhifan Luo, Shuo Shao, Su Zhang, Lijing Zhou, Yuke Hu, Chenxu Zhao, Zhihao Liu, and Zhan Qin. Shadow in the cache: Unveiling and mitigating privacy risks of KV-cache in LLM inference. In *Network and Distributed System Security Symposium (NDSS)*, 2026.

[16] NVIDIA. NVIDIA Dynamo: Request extensions (nvext), cache salt and tenant isolation. `https://docs.nvidia.com/dynamo/v1.3.0/additional-resources/nvidia-request-extensions-nvext.md`. Accessed October 2026.

[17] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024.

[18] Panagiotis Georgios Pennas, Konstantinos Papaioannou, Marco Guarnieri, and Thaleia Dimitra Doudali. CacheSolidarity: Preventing prefix caching side channels in multi-tenant LLM serving systems. arXiv:2603.10726, 2026. v2 retitled PrefixWall; artifact released for Middleware '26 – VERIFY final venue and title.

[19] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng,

and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170. USENIX Association, 2025.

[20] Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds. In *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*, 2009. VERIFY pages.

[21] L. Song, Z. Pang, W. Wang, Z. Wang, X. Wang, H. Chen, W. Song, Y. Jin, D. Meng, and R. Hou. The early bird catches the leak: Unveiling timing side channels in LLM serving systems. arXiv:2409.20002, 2024. VERIFY: a later version is cited elsewhere as IEEE TIFS 2025.

[22] He Sun, Shinan Liu, Siyuan Ma, Junhao Li, Mingjun Xiao, and Wenhao Jiang. Agent-assisted side-channel attacks on non-prefix KV cache in RAG. arXiv:2606.21842, 2026.

[23] vLLM Project. Automatic prefix caching and per-request `cache_salt`. `https://docs.vllm.ai/`. Accessed October 2026. VERIFY exact page URL.

[24] vLLM Project. CVE-2026-105752: Harmony tool continuations drop `cache_salt`, restoring a cross-tenant prefix-cache membership oracle. GitHub Security Advisory GHSA-935w-9g4m-p28p, 2026. Fixed in vLLM 0.30.0.

[25] G. Wu, Z. Zhang, Y. Zhang, W. Wang, J. Niu, Y. Wu, and Y. Zhang. I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving. In *Network and Distributed System Security Symposium (NDSS)*, 2025. VERIFY: expand initials to full names.

[26] XiangFan Wu, Lingyun Ying, Guoqiang Chen, Yacong Gu, and Haipeng Qu. Cache me, catch you: Cache related security threats in LLM serving frameworks. In *Network and Distributed System Security Symposium (NDSS)*, 2026.

[27] Yuji Yamamoto and Satoshi Matsuura. Bit-flip vulnerability of shared KV-cache blocks in LLM serving systems. arXiv:2604.17249, 2026.

[28] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, pages 94–109, 2025.

[29] Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 719–732, 2014.

[30] Danfeng Zhang, Aslan Askarov, and Andrew C. Myers. Predictive mitigation of timing channels in interactive systems. In *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)*, 2011. VERIFY pages.

[31] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, et al. Efficiently programming large language models using SGLang. arXiv:2312.07104, 2023. VERIFY: cite the NeurIPS 2024 version if preferred.

[32] X. Zheng, H. Han, S. Shi, Q. Fang, Z. Du, X. Hu, and Q. Guo. InputSnatch: Stealing input in LLM services via timing side-channel attacks. arXiv:2411.18191, 2024. VERIFY: expand initials to full names.

[33] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

A Ethics Considerations

[TODO: Follow the USENIX Security '27 ethics guidance (expect the same for '28). Draft points:]

- **Stakeholders.** API users whose prompts could leak; operators of Mooncake-style deployments; maintainers of vLLM, SGLang, LMCache and Mooncake.
- **Own infrastructure.** All attacks run on our testbed against synthetic victims. No real user data is processed.
- **Public services.** Any measurement of public APIs is limited to observing our own requests' metadata (e.g., whether usage records report cached tokens), never to inferring other customers' data. [TODO: Decide whether to do this at all.]
- **Disclosure.** We will notify **[the Mooncake maintainers and affected engine projects]** at least **[90]** days before submission, with the attack harness and suggested mitigations.
- **Release.** The defense is released openly; the attack harness is released after disclosure windows close.

B Open Science

USENIX Security requires an Open Science appendix describing every artifact and how reviewers can access it. [TODO: Finalize URLs (anonymized for review).]

- MIRAGE source (Conductor, VCD, release controller, scrubber), as patches against public vLLM and Mooncake versions.
- Attack harness and classifiers for **A1**–**A6**.
- Principal-labelled derivatives of Mooncake's public traces, with the labelling script.
- Scripts that regenerate every figure and table from raw logs.
- Hardware note: full-scale runs need RDMA-connected multi-GPU nodes; we provide a single-node configuration that reproduces the security results at reduced scale.